

PetalLogic Final Implementation plan

PART 1 — BLOCK 0: UNIFIED LAUNCHPAD (FOUNDATION)

(Paste this first in your Google Doc)

BLOCK 0: THE UNIFIED LAUNCHPAD

Goal

Before writing any business logic:

- Your **environment must be perfect**
- Your **project structure must be clean**
- Your **services must be runnable**

If this block is wrong → everything breaks later.

TASK 0.1 — Environment Setup (Both)

Step 1 — Install Java 21

Run:

```
java --version
```

Expected Output

```
openjdk version "21"
```

If Wrong

- Install JDK 21 from official source
 - Set JAVA_HOME properly
-

✅ Step 2 — Install Docker

Run:

```
docker --version
```

✅ Step 3 — Pull Required Containers

```
docker pull mysql
```

```
docker pull redis
```

✅ Step 4 — Install Ollama (AI Engine)

Install and run:

```
ollama pull qwen2.5:7b
```

✅ Step 5 — Verify AI is Working

```
ollama run qwen2.5:7b
```

Type:

Explain binary search

✓ You must get response

🧩 TASK 0.2 — Project Creation (Pavan)

✅ Step 1 — Create Root Folder

```
petallogic-ai/
```

✅ Step 2 — Create Parent **pom.xml**

```
<project>
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.petallogic</groupId>
<artifactId>petallogic-ai</artifactId>
<version>1.0.0</version>

<packaging>pom</packaging>

<modules>
  <module>common-lib</module>
  <module>eureka-server</module>
  <module>api-gateway</module>
  <module>auth-service</module>
  <module>ai-service</module>
  <module>topic-service</module>
  <module>progress-service</module>
</modules>
</project>
```

✓ Step 3 — Add Dependency Management

```
<dependencyManagement>
  <dependencies>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>3.4.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>

    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2024.0.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>

  </dependencies>
</dependencyManagement>
```

🧩 TASK 0.3 — Monorepo Modules (Pavan)

✓ Step 1 — Create Modules

Inside root:

common-lib/
eureka-server/
api-gateway/
auth-service/
ai-service/
topic-service/
progress-service/

✅ Step 2 — Convert Each Into Spring Boot Project

Each service must have:

- `@SpringBootApplication`
 - Separate `application.yml`
-

! Important Rule

- `common-lib` → NO main class
 - All others → runnable services
-

🧩 TASK 0.4 — Docker Setup (Kruthika)

✅ Step 1 — Create `docker-compose.yml`

version: "3.8"

services:

```
mysql:
  image: mysql:8
  container_name: petal-mysql
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: petallogic
  ports:
    - "3306:3306"
```

```
redis:
  image: redis:7
  container_name: petal-redis
```

ports:

- "6379:6379"

ollama:

image: ollama/ollama

container_name: petal-ollama

ports:

- "11434:11434"

✅ Step 2 — Run Containers

docker-compose up -d

✅ Step 3 — Verify

docker ps

✓ You must see:

- mysql
 - redis
 - ollama
-

🧩 TASK 0.5 — Git Setup (Both)

✅ Step 1

git init

git checkout -b develop

✅ Step 2 — Create **.gitignore**

target/

*.log

.env

✅ Step 3 — First Commit

git add .

git commit -m "INIT: Base Project Setup"



BLOCK 0 DONE CRITERIA

- ✓ Java 21 installed
 - ✓ Docker running
 - ✓ Ollama responding
 - ✓ Project structure created
 - ✓ All modules exist
 - ✓ Git initialized
-



COMMON MISTAKES

- Using Java 17 → ✗ (Virtual Threads won't work)
 - Skipping Docker → ✗ later failures
 - Wrong module structure → ✗ build breaks
 - Not verifying Ollama → ✗ AI fails later
-



PART 2 — BLOCK 1: DISTRIBUTED SKELETON (EUREKA + GATEWAY + SERVICE REGISTRATION)



BLOCK 1: THE DISTRIBUTED SKELETON



Goal

By the end of this block:

- Eureka Server must be running
- All services must register inside Eureka
- API Gateway must route requests properly
- You must access everything ONLY through Gateway



TASK 1.1 — EUREKA SERVER (Pavan)

Step 1 — Create Service

Create a new Spring Boot project:

- Name: eureka-server
 - Dependencies:
 - Spring Web
 - Eureka Server
-

Step 2 — Add Dependency

Add this to pom.xml:

Step 3 — Main Class

Create main class:

```
@EnableEurekaServer
@SpringBootApplication
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Step 4 — application.yml

```
server:
  port: 8761
```

```
spring:
  application:
    name: eureka-server
```

eureka:
client:
register-with-eureka: false
fetch-registry: false

Step 5 — Enable Virtual Threads

Add in application.properties:

spring.threads.virtual.enabled=true

Step 6 — Run and Verify

Open browser:

<http://localhost:8761>

✓ You must see Eureka dashboard

Common Issues

- Port already in use → change port
 - Dependency missing → app fails to start
-



TASK 1.2 — API GATEWAY (Kruthika)

Step 1 — Create Service

Create Spring Boot project:

- Name: api-gateway
 - Dependencies:
 - Spring Cloud Gateway
 - Eureka Client
 - WebFlux
-

Step 2 — Add Dependencies

Step 3 — Configure Routing

application.yml:

server:

port: 8080

spring:

application:

name: api-gateway

cloud:

gateway:

routes:

- id: auth-service
 - uri: lb://AUTH-SERVICE
 - predicates:
 - Path=/auth/**
- id: ai-service
 - uri: lb://AI-SERVICE
 - predicates:
 - Path=/ai/**
- id: topic-service
 - uri: lb://TOPIC-SERVICE
 - predicates:
 - Path=/topics/**

eureka:

client:

service-url:

defaultZone: <http://localhost:8761/eureka>

Step 4 — Run Gateway

Start service → should run on port 8080

Important Rule

Never call services directly.

Always call through:

<http://localhost:8080>



TASK 1.3 — SERVICE REGISTRATION (Pavan)

Step 1 — Add Dependency to ALL Services

Step 2 — Add Config to ALL Services

application.yml:

spring:

application:

name: auth-service (change per service)

eureka:

client:

service-url:

defaultZone: <http://localhost:8761/eureka>

Step 3 — Run Order (Important)

Start in this order:

1. Eureka Server
 2. API Gateway
 3. Any service (auth-service first)
-

Step 4 — Verify

Open:

<http://localhost:8761>

✓ You must see:

- API-GATEWAY
 - AUTH-SERVICE
-

TASK 1.4 — COMMON RESPONSE DTO (Pavan)

Step 1 — Create in common-lib

Create class:

```
public record ApiResponse(  
    boolean success,  
    String message,  
    T data,  
    LocalDateTime timestamp  
) {}
```

Step 2 — Use in Controller

Example:

```
return new ApiResponse<>(  
    true,  
    "Success",  
    data,  
    LocalDateTime.now()  
);
```

TASK 1.5 — GLOBAL ERROR HANDLING (Kruthika)

Step 1 — Create Global Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ProblemDetail handleException(Exception ex) {
        return ProblemDetail.forStatus(500);
    }

}
```

Step 2 — Extend Later

Add:

- 400 Bad Request
 - 401 Unauthorized
 - 403 Forbidden
 - 429 Too Many Requests
-



TASK 1.6 — END-TO-END TEST (Both)

Step 1 — Create Test API (auth-service)

```
@RestController
@RequestMapping("/auth")
public class TestController {

    @GetMapping("/test")
    public String test() {
        return "Auth Service Working";
    }

}
```

Step 2 — Call via Gateway

Open:

Expected Output

Auth Service Working



BLOCK 1 DONE CHECKLIST

- ✓ Eureka dashboard working
 - ✓ Gateway running on 8080
 - ✓ Services registered in Eureka
 - ✓ Routing working through Gateway
 - ✓ ApiResponse used
 - ✓ Global error handler working
-



COMMON MISTAKES

- Calling service directly instead of gateway
 - Wrong service name (must match EXACT)
 - Forgetting Eureka config
 - Gateway routes not configured properly
-



PART 3 — BLOCK 2: SECURITY (ZERO-TRUST IMPLEMENTATION)



BLOCK 2: IDENTITY & ZERO-TRUST SECURITY



By the end of this block:

- Users can **register & login**
 - Passwords are **securely hashed (Argon2)**
 - JWT tokens are generated using **RS256 (asymmetric encryption)**
 - Gateway validates every request
 - All services trust **ONLY the Gateway**
-



ARCHITECTURE (VERY IMPORTANT UNDERSTAND FIRST)

Flow:

1. User → Login
2. Auth Service → Generates JWT (using PRIVATE key)
3. Client → Sends JWT in header
4. Gateway → Validates JWT (using PUBLIC key)
5. Gateway → Injects userId into header
6. Internal services → Trust Gateway



No service trusts the client directly



Only Gateway is trusted



TASK 2.1 — RSA KEY GENERATION (Pavan)

Step 1 — Generate Keys

Run in terminal:

```
openssl genrsa -out private.pem 2048  
openssl rsa -in private.pem -pubout > public.pem
```

Step 2 — Place Keys

Put:

- private.pem → inside auth-service
- public.pem → inside api-gateway

Step 3 — Convert to String (Important)

Read file content and store in application.yml (optional but recommended later)

TASK 2.2 — USER ENTITY + DATABASE (Pavan)

Step 1 — Configure MySQL (auth-service)

application.yml:

```
spring:
datasource:
url: jdbc:mysql://localhost:3306/petallogic
username: root
password: root

jpa:
hibernate:
ddl-auto: update
show-sql: true
```

Step 2 — Create User Entity

```
@Entity
public class User {

@Id
@GeneratedValue
private UUID id;

private String email;
private String password;
private String codingLevel;

}
```

Step 3 — Create Repository

```
public interface UserRepository extends JpaRepository<User, UUID> {  
    Optional findByEmail(String email);  
}
```

TASK 2.3 — PASSWORD SECURITY (Argon2) (Kruthika)

Step 1 — Create Password Encoder

```
@Bean  
public PasswordEncoder passwordEncoder() {  
    return new Argon2PasswordEncoder();  
}
```

Step 2 — Hash Password on Register

```
user.setPassword(passwordEncoder.encode(request.getPassword()));
```

Step 3 — Validate Password on Login

```
passwordEncoder.matches(rawPassword, encodedPassword)
```

Important

- ✗ Never store plain password
 - ✓ Always hash using Argon2
-



TASK 2.4 — JWT ENGINE (RS256) (Pavan)

Step 1 — Add Dependency

Step 2 — JWT Service

```
public class JwtService {  
  
    public String generateToken(UUID userId) {  
        return Jwts.builder()  
            .subject(userId.toString())  
            .signWith(privateKey)  
            .compact();  
    }  
  
}
```

Step 3 — Extract User ID

```
public String extractUser(String token) {  
    return Jwts.parser()  
        .verifyWith(publicKey)  
        .build()  
        .parseSignedClaims(token)  
        .getPayload()  
        .getSubject();  
}
```



TASK 2.5 — AUTH APIs (Both)

Step 1 — Register API

POST /auth/register

Flow:

1. Receive email + password
 2. Hash password
 3. Save user
-

Step 2 — Login API

POST /auth/login

Flow:

1. Validate user exists
 2. Check password
 3. Generate JWT
 4. Return token
-

Sample Response

```
{  
  "token": "eyJhbGciOiJSUzI1NiIs..."  
}
```



TASK 2.6 — GATEWAY JWT FILTER (Kruthika)

Step 1 — Create Filter

```
@Component  
public class JwtFilter implements GlobalFilter {  
  
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {  
  
        String token = extractFromHeader(exchange);  
  
        if (token invalid) → reject  
  
        String userId = validateToken(token);  
  
        addHeader(exchange, "X-User-ID", userId);  
  
        return chain.filter(exchange);  
    }  
}
```

}

}

Step 2 — Extract Token

Authorization: Bearer

Step 3 — Reject Invalid Requests

Return:

- 401 Unauthorized
-



TASK 2.7 — SECURE ROUTES

Rule

- /auth/** → public
 - everything else → protected
-

Implement in Gateway

If path starts with /auth → skip filter

Else → validate JWT



TASK 2.8 — TEST SECURITY (Both)

Step 1 — Register User

POST /auth/register

Step 2 — Login

POST /auth/login

✓ Get token

Step 3 — Call Protected API

Add header:

Authorization: Bearer

Expected

✓ Request works

Without token:

✗ Must fail (401)



BLOCK 2 DONE CHECKLIST

- ✓ User saved in DB
 - ✓ Password hashed
 - ✓ JWT generated
 - ✓ Gateway validates token
 - ✓ Header injected
 - ✓ Unauthorized blocked
-



COMMON MISTAKES

- Using symmetric JWT → ✗
- Storing raw password → ✗
- Not validating token in gateway → ✗

- Forgetting "Bearer " prefix → ❌
-

PART 4 — BLOCK 3: AI SOCRATIC ENGINE (FULL IMPLEMENTATION)

BLOCK 3: THE AI SOCRATIC ENGINE

Goal

By the end of this block:

- Your backend can **talk to local AI (Ollama)**
 - AI returns **structured JSON (not random text)**
 - AI behaves like a **Socratic tutor (guides, not solves)**
 - Responses can be **streamed (real-time thinking effect)**
-

VERY IMPORTANT PRINCIPLE

👉 AI should **NOT** give code

👉 AI should:

- Identify mistakes
 - Guide thinking
 - Give hints
 - Show logical steps
-

TASK 3.1 — AI SERVICE SETUP (Pavan)

Step 1 — Add Dependency

In ai-service:

Add Spring AI dependency (Ollama support)

Step 2 — Configure Connection

application.yml:

```
spring:
  ai:
    ollama:
      base-url: http://localhost:11434
      chat:
        model: qwen2.5:7b
```

Step 3 — Create AI Client

Create a service class:

```
@Service
public class AIClientService {

    private final ChatClient chatClient;

    public AIClientService(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }

    public String askAI(String prompt) {
        return chatClient.prompt(prompt).call().content();
    }

}
```

Verify

Call AI once with simple prompt

✓ Must return response

TASK 3.2 — AI CONTROLLER (Kruthika)

Step 1 — Create Endpoint

POST /ai/analyze

Step 2 — Request Body

Create DTO:

```
public class AIRequest {  
    private String problem;  
    private String userInput;  
}
```

Step 3 — Controller

```
@RestController  
@RequestMapping("/ai")  
public class AIController {  
  
    private final AIClientService aiService;  
  
    @PostMapping("/analyze")  
    public String analyze(@RequestBody AIRequest request) {  
        return aiService.askAI(request.getUserInput());  
    }  
  
}
```

Test

Call endpoint → AI must respond

TASK 3.3 — SYSTEM PROMPT DESIGN (Both — MOST IMPORTANT)

Goal

Control AI behavior strictly

Step 1 — Create System Prompt

"

You are an expert DSA tutor.
Do NOT give full code solutions.
Identify logical mistakes in user's thinking.
Give hints, not answers.
Break solution into steps.
Return response in JSON format only.

"

Step 2 — Combine Prompt

finalPrompt = systemPrompt + userInput

RULE

If prompt is weak → AI becomes useless

TASK 3.4 — STRUCTURED OUTPUT (Kruthika)

Goal

Force AI to return JSON (not text)

Step 1 — Define Format

AI must return:

```
{  
  "logicalGaps": [...],  
  "hint": "...",  
  "steps": ["step1", "step2"],  
  "flowchartNodes": ["node1", "node2"]  
}
```

Step 2 — Validate Response

- Check JSON format
 - If invalid → retry
-

Step 3 — Parsing

Convert response into Java object



TASK 3.5 — AI PROCESSING ENGINE (Pavan)

Step 1 — Full Flow

1. Receive request
 2. Build prompt
 3. Send to AI
 4. Receive response
 5. Parse JSON
 6. Return structured output
-

Step 2 — Add Retry Logic

If AI fails:

- Retry 2 times

- If still fails → return fallback
-

Step 3 — Timeout Protection

Avoid long waits (very important)



TASK 3.6 — STREAMING RESPONSE (Both)



Goal

Show AI thinking in real-time

Step 1 — Use Flux

```
public Flux streamResponse(String prompt) {  
    return chatClient.prompt(prompt).stream().content();  
}
```

Step 2 — Controller

```
@GetMapping("/stream")  
public Flux stream(...)
```

Step 3 — Frontend receives chunks



Result

User sees response like:

"Thinking..."

"Analyzing..."

"Here is your hint..."



TASK 3.7 — CONTEXT INJECTION (Both)



Goal

Make AI smarter using history

Step 1

Fetch user past mistakes

Step 2

Add to prompt:

"User struggles with recursion"

Result

AI becomes personalized



TASK 3.8 — TESTING (Both)

Test Cases

1. Valid input
 2. Empty input
 3. Wrong logic input
 4. Edge case
-

Expected

- ✓ JSON response
 - ✓ No code leakage
 - ✓ Helpful hints
-



BLOCK 3 DONE CHECKLIST

- ✓ AI connected
 - ✓ Prompt working
 - ✓ JSON output correct
 - ✓ Streaming working
 - ✓ Context injection working
-



COMMON MISTAKES

- AI giving full code → ✗
 - Not enforcing JSON → ✗
 - No retry logic → ✗
 - Blocking calls → ✗
-



PART 5 — BLOCK 4: LOGIC VISUALIZATION (GRAPH ENGINE)



BLOCK 4: LOGIC VISUALIZATION ENGINE



Goal

By the end of this block:

- AI output is converted into **graph structure**
- Each logical step becomes a **node**
- Steps are connected as a **flow**
- Backend sends **ready-to-render graph JSON**

👉 This powers your frontend (React Flow / diagrams)



CORE IDEA

AI gives:

- Steps
- Logical flow

You convert it into:

- Nodes
 - Edges
-



TASK 4.1 — GRAPH DATA STRUCTURE (Pavan)



Goal

Define how graph will look in backend

Step 1 — Create GraphNode

```
public class GraphNode {  
  
    private String id;  
    private String label;  
    private String type;  
  
}
```

Step 2 — Create GraphEdge

```
public class GraphEdge {
```

```
private String id;  
private String source;  
private String target;  
  
}
```

Step 3 — Create GraphResponse

```
public class GraphResponse {  
  
private List<GraphNode> nodes;  
private List<GraphEdge> edges;  
  
}
```

Important

- id must be unique
 - label = step text
 - type = optional (start, process, decision)
-



TASK 4.2 — AI STEPS → NODES (Kruthika)



Goal

Convert AI response steps into nodes

Input Example (from AI)

steps:

- "Check input size"
- "Initialize variables"
- "Loop through array"

Step 1 — Create Mapper

```
public List mapStepsToNodes(List steps) {  
  
    List<GraphNode> nodes = new ArrayList<>();  
  
    for (int i = 0; i < steps.size(); i++) {  
  
        GraphNode node = new GraphNode();  
        node.setIdx("node-" + i);  
        node.setLabel(steps.get(i));  
        node.setType("process");  
  
        nodes.add(node);  
    }  
  
    return nodes;  
  
}
```

Result

Each step becomes a node



TASK 4.3 — CREATE EDGES (Pavan)



Goal

Connect nodes sequentially

Step 1 — Link Nodes

```
public List createEdges(List nodes) {  
  
    List<GraphEdge> edges = new ArrayList<>();
```

```
for (int i = 0; i < nodes.size() - 1; i++) {  
  
    GraphEdge edge = new GraphEdge();  
    edge.setIdx("edge-" + i);  
    edge.setSource(nodes.get(i).getId());  
    edge.setTarget(nodes.get(i + 1).getId());  
  
    edges.add(edge);  
}  
  
return edges;  
  
}
```

Result

node-0 → node-1 → node-2

TASK 4.4 — BUILD GRAPH RESPONSE (Pavan)

Step 1 — Combine

```
public GraphResponse buildGraph(List steps) {  
  
    List<GraphNode> nodes = mapStepsToNodes(steps);  
    List<GraphEdge> edges = createEdges(nodes);  
  
    return new GraphResponse(nodes, edges);  
  
}
```

TASK 4.5 — API ENDPOINT (Kruthika)

Step 1 — Create Endpoint

GET /ai/flow/{sessionId}

Step 2 — Flow

1. Fetch AI response
 2. Extract steps
 3. Convert to graph
 4. Return GraphResponse
-

Sample Response

```
{
  "nodes": [
    { "id": "node-0", "label": "Start", "type": "start" }
  ],
  "edges": [
    { "source": "node-0", "target": "node-1" }
  ]
}
```



TASK 4.6 — ENHANCEMENTS (Both)

Add Node Types

- start
 - process
 - decision
 - end
-

Example Logic

if step contains "if" → decision node

Add Position (Optional for frontend)

x, y coordinates

TASK 4.7 — VALIDATION (Both)

Test Cases

1. Empty steps
 2. Single step
 3. Multiple steps
-

Expected

- ✓ Graph always valid
 - ✓ No broken edges
 - ✓ No duplicate IDs
-

TASK 4.8 — STORE GRAPH (Optional Advanced)

Store graph in DB

So user can revisit flow later

BLOCK 4 DONE CHECKLIST

- ✓ Graph DTO created
- ✓ Steps → nodes mapping working
- ✓ Nodes linked correctly

- ✓ API returning graph
 - ✓ Valid JSON structure
-



COMMON MISTAKES

- Duplicate node IDs → ✗
 - Missing edges → ✗
 - Empty steps crash → ✗
 - Not validating AI output → ✗
-



PART 6 — BLOCK 5: CURRICULUM & DATA ENGINE



BLOCK 5: CURRICULUM & DATA ENGINE



Goal

By the end of this block:

- Problems are stored in database
 - Each problem has structured data (title, difficulty, tags, solution)
 - You can **filter, search, and paginate**
 - Initial dataset is auto-loaded using Flyway
-



CORE DESIGN DECISION

- 👉 Problems are NOT just plain text
- 👉 They must support:

- Structured solutions
 - Tags
 - Future AI extensions
-



TASK 5.1 — DATABASE DESIGN (Pavan)

Step 1 — Create Problem Entity

```
@Entity
public class Problem {

    @Id
    @GeneratedValue
    private Long id;

    private String title;
    private String description;
    private String difficulty;

    @Column(columnDefinition = "json")
    private String solutionJson;

}
```

Important

solutionJson stores:

- Optimal approach
 - Steps
 - Explanation
-

Step 2 — Example JSON

```
{
  "approach": "Two pointers",
  "steps": ["Initialize", "Loop", "Compare"],
  "complexity": "O(n)"
}
```



TASK 5.2 — ADD TAGS SUPPORT (Pavan)

Step 1 — Create Tag Field

Option 1 (simple):

```
private String tags;
```

Example:

```
"array,twopointer,medium"
```

Better Option (recommended):

```
@ElementCollection
```

```
private List tags;
```



TASK 5.3 — REPOSITORY (Pavan)

```
public interface ProblemRepository extends JpaRepository<Problem, Long>, JpaSpecificationExecutor {  
}
```



TASK 5.4 — FILTERING (Kruthika)



Goal

User should filter problems like:

- difficulty = easy
 - tag = array
-

Step 1 — Create Specification

```
public class ProblemSpecification {
```

```
public static Specification<Problem> hasDifficulty(String difficulty) {  
    return (root, query, cb) -> cb.equal(root.get("difficulty"), difficulty);  
}  
  
}
```

Step 2 — Combine Filters

```
Specification spec = Specification.where(null);  
  
if (difficulty != null)  
    spec = spec.and(hasDifficulty(difficulty));
```



TASK 5.5 — SERVICE LAYER (Pavan)

```
public List getProblems(String difficulty) {  
    return problemRepository.findAll(spec);  
}
```



TASK 5.6 — CONTROLLER (Kruthika)

GET /topics

Query params:

- difficulty
 - tag
-

Example:

/topics?difficulty=easy

Controller

TASK 5.7 — PAGINATION (Kruthika)

Step 1 — Add Pageable

Pageable pageable

Step 2 — Return Page

Page

Example

/topics?page=0&size=10

TASK 5.8 — FLYWAY MIGRATION (Pavan)

Goal

Auto-load problems when app starts

Step 1 — Add Dependency

Flyway

Step 2 — Create Migration File

V1__init.sql

Step 3 — Insert Data

INSERT INTO problem (...) VALUES (...)

Step 4 — Run App

✓ Data auto-loaded



TASK 5.9 — SEARCH (Kruthika)



Goal

Search by title

Step 1

Add method:

findByTitleContaining(String keyword)

Example

/topics/search?query=array



TASK 5.10 — VALIDATION (Both)

Test Cases

- No filters
 - Difficulty filter
 - Tag filter
 - Pagination
 - Search
-

Expected

- ✓ Fast responses
 - ✓ Correct filtering
 - ✓ No errors
-

TASK 5.11 — PERFORMANCE OPTIMIZATION (Pavan)

Add Index

```
@Index(name = "idx_difficulty", columnList = "difficulty")
```

Why?

Faster queries

TASK 5.12 — CACHE (Optional Intro)

Cache frequently used problems in Redis



BLOCK 5 DONE CHECKLIST

- ✓ Problem entity created
 - ✓ JSON solution stored
 - ✓ Filtering working
 - ✓ Pagination working
 - ✓ Search working
 - ✓ Flyway seeding working
-



COMMON MISTAKES

- Storing plain text only → ✗
 - No pagination → ✗
 - No indexing → ✗
 - Hardcoding data → ✗
-



PART 7 — BLOCK 6: CONTEXTUAL MEMORY (AI MEMORY SYSTEM)



BLOCK 6: CONTEXTUAL MEMORY



Goal

By the end of this block:

- Every AI interaction is **stored**
 - Active session is **cached in Redis**
 - AI gets **user history injected into prompts**
 - System becomes **personalized per user**
-



CORE IDEA

Without memory → AI is generic

With memory → AI becomes **personal tutor**

TASK 6.1 — SESSION DATABASE DESIGN (Pavan)

Step 1 — Create Session Entity

```
@Entity
public class AISession {

    @Id
    @GeneratedValue
    private Long id;

    private UUID userId;

    @Column(length = 5000)
    private String userInput;

    @Column(length = 5000)
    private String aiResponse;

    private LocalDateTime createdAt;

}
```

Why this is important

- Stores entire conversation
 - Enables history
 - Enables analytics later
-

Step 2 — Repository

```
public interface AISessionRepository extends JpaRepository<AISession, Long> {
```

```
List<AISession> findTop5ByUserIdOrderByCreatedAtDesc(UUID userId);
```

```
}
```

TASK 6.2 — SAVE AI INTERACTIONS (Pavan)

Step 1 — After AI Response

Inside AI service:

```
AISession session = new AISession();  
session.setUserId(userId);  
session.setUserInput(request.getUserInput());  
session.setAiResponse(aiResponse);  
session.setCreatedAt(LocalDateTime.now());  
  
sessionRepository.save(session);
```

Important

Save EVERY interaction

TASK 6.3 — FETCH HISTORY (Pavan)

Step 1 — Get Recent Sessions

```
List history = sessionRepository  
.findTop5ByUserIdOrderByCreatedAtDesc(userId);
```

Step 2 — Extract Key Info

From history:

- previous mistakes
 - previous topics
-

TASK 6.4 — REDIS ACTIVE SESSION **(Kruthika)**

Goal

Fast access to current session

Step 1 — Add Redis Dependency

spring-boot-starter-data-redis

Step 2 — Configure Redis

spring:
redis:
host: localhost
port: 6379

Step 3 — Store Active Session

```
redisTemplate.opsForValue().set(  
    "session:" + userId,  
    currentSession  
);
```

Step 4 — Set Expiry

Set TTL (e.g., 30 minutes)

Why Redis?

- Fast retrieval
 - Reduces DB load
-

TASK 6.5 — CONTEXT INJECTION (Both — VERY IMPORTANT)

Goal

Use history to improve AI responses

Step 1 — Extract Weakness

From history:

- repeated mistakes
 - failed logic
-

Step 2 — Inject into Prompt

Add to system prompt:

"User previously struggled with recursion and loop logic."

Step 3 — Final Prompt

System Prompt + User Input + History Context

Result

AI becomes:

✓ Personalized

- ✓ Smarter
 - ✓ Consistent
-

TASK 6.6 — HISTORY API (Kruthika)

Step 1 — Create Endpoint

GET /history

Step 2 — Fetch Data

Use userId from header

Step 3 — Return Response

List of past sessions

TASK 6.7 — CLEANUP STRATEGY (Pavan)

Problem

DB grows infinitely

Solution

- Delete old sessions (optional)
 - Archive data
-

TASK 6.8 — TESTING (Both)

Test Cases

1. New user (no history)
 2. Existing user
 3. Multiple sessions
 4. Redis hit vs DB hit
-

Expected

- ✓ History fetched correctly
 - ✓ Redis fast response
 - ✓ AI uses history
-

TASK 6.9 — EDGE CASE HANDLING

- Empty history → skip injection
 - Corrupt session → ignore
 - Large history → limit to 5
-

BLOCK 6 DONE CHECKLIST

- ✓ Sessions saved
 - ✓ History fetched
 - ✓ Redis working
 - ✓ Context injected into AI
 - ✓ API working
-

COMMON MISTAKES

- Not saving sessions → 

- Injecting too much history → ❌
 - No Redis TTL → ❌
 - Blocking DB queries → ❌
-

PART 8 — BLOCK 7: ANALYTICS & STREAK ENGINE

BLOCK 7: ANALYTICS & GAMIFICATION

Goal

By the end of this block:

- You track **daily user activity**
- You generate a **GitHub-style heatmap**
- You maintain **streaks**
- You implement an **XP + leveling system**

👉 This directly drives **user retention**

CORE IDEA

User should feel:

- “I don’t want to break my streak”
 - “I am improving daily”
 - “I am leveling up”
-

TASK 7.1 — USER ACTIVITY TRACKING (Pavan)

Step 1 — Create Activity Entity

```
@Entity
public class UserActivity {

    @Id
    @GeneratedValue
    private Long id;

    private UUID userId;
    private LocalDate date;
    private int problemsSolved;

}
```

Step 2 — Repository

```
public interface UserActivityRepository extends JpaRepository<UserActivity, Long> {

    List<UserActivity> findByUserId(UUID userId);

}
```

Step 3 — Update Activity

Whenever user solves problem:

- Find today's record
 - If exists → increment
 - Else → create new
-



TASK 7.2 — HEATMAP DATA (Pavan)



Goal

Return 365-day activity map

Step 1 — Query Data

Fetch all activity for user

Step 2 — Convert to Response

Create:

```
{  
  "date": "2026-04-30",  
  "intensity": 3  
}
```

Step 3 — Logic

- 0 → no activity
 - 1–2 → low
 - 3–5 → medium
 - 5+ → high
-

Step 4 — API

GET /analytics/heatmap

TASK 7.3 — STREAK CALCULATION (Kruthika)

Goal

Track consecutive active days

Step 1 — Logic

1. Get today's date
 2. Check if solved today
 3. Check yesterday
 4. Continue backward
-

Step 2 — Example

Day1 ✓

Day2 ✓

Day3 ✗

→ streak = 2

Step 3 — API

GET /profile/streak

Response

```
{  
  "currentStreak": 5  
}
```



TASK 7.4 — XP SYSTEM (Pavan)



Goal

Reward learning progress

Step 1 — Define Rules

- Solve problem → +10 XP

- Correct logic → +20 XP
 - AI match → +30 XP
-

Step 2 — Store XP

Add in User entity:

```
private int xp;
```

Step 3 — Level Logic

```
level = xp / 100
```

Step 4 — API

GET /profile/xp

Response

```
{  
  "xp": 240,  
  "level": 2  
}
```



TASK 7.5 — DAILY UPDATE FLOW (Both)

When user solves problem:

1. Update UserActivity
 2. Update XP
 3. Update streak
-

Important

All must happen in one transaction

TASK 7.6 — PERFORMANCE OPTIMIZATION (Pavan)

Use Native Query

For heatmap aggregation

Why?

Faster than JPA

TASK 7.7 — CACHING (Kruthika)

Cache:

- heatmap
- streak

Using Redis

TASK 7.8 — TESTING (Both)

Test Cases

- New user
- Continuous usage

- Missed day
 - Large data
-

Expected

- ✓ Correct streak
 - ✓ Correct XP
 - ✓ Accurate heatmap
-

TASK 7.9 — EDGE CASES

- User inactive for long
 - Timezone issues
 - Multiple submissions same day
-

BLOCK 7 DONE CHECKLIST

- ✓ Activity tracked
 - ✓ Heatmap working
 - ✓ Streak correct
 - ✓ XP system working
 - ✓ APIs returning correct data
-

COMMON MISTAKES

- Not handling timezone → 
 - Wrong streak logic → 
 - No caching → 
 - XP miscalculation → 
-

PART 9 — BLOCK 8: SMART TUTOR PERSONALIZATION

BLOCK 8: SMART TUTOR PERSONALIZATION

Goal

By the end of this block:

- AI adapts to **user skill level**
 - AI changes **tone, depth, and explanation style**
 - System identifies **user weaknesses automatically**
 - Backend exposes a **Weakness Dashboard**
-

CORE IDEA

Same problem → different explanation based on user:

- Beginner → simple + analogies
 - Intermediate → structured + hints
 - Pro → minimal guidance + edge cases
-

TASK 8.1 — USER LEVEL DESIGN (Pavan)

Step 1 — Extend User Entity

Add:

```
private String codingLevel;
```

Step 2 — Possible Values

- BEGINNER
 - INTERMEDIATE
 - PRO
-

Step 3 — Default Level

On register:

```
user.setCodingLevel("BEGINNER");
```

TASK 8.2 — PROMPT TAILORING (Kruthika + Pavan)

Goal

Modify AI prompt dynamically

Step 1 — Define Prompt Variants

BEGINNER:

"Explain in simple terms. Use real-world examples. Avoid technical jargon."

INTERMEDIATE:

"Explain with step-by-step logic and reasoning."

PRO:

"Be concise. Focus on optimization and edge cases."

Step 2 — Inject into Prompt

```
finalPrompt =  
systemPrompt +  
levelPrompt +  
userInput
```

Result

AI behaves differently per user

TASK 8.3 — WEAKNESS DETECTION ENGINE (Pavan)

Goal

Identify what user struggles with

Step 1 — Analyze Sessions

From AI history:

- logical gaps
 - repeated mistakes
-

Step 2 — Aggregate Data

Example:

recursion → 5 failures

loops → 2 failures

Step 3 — Store Result

Create:

UserWeakness entity

private String topic;
private int score;

TASK 8.4 — UPDATE WEAKNESS (Pavan)

Step 1 — After Each AI Response

If AI detects gap:

- increment weakness score
-

Step 2 — Logic

if gap contains "recursion"
→ update recursion score

TASK 8.5 — WEAKNESS API (Kruthika)

Step 1 — Create Endpoint

GET /profile/weakness

Step 2 — Response

```
{  
  "recursion": 80,  
  "loops": 40  
}
```

Meaning

Higher score → weaker area

TASK 8.6 — AI CONTEXT ENHANCEMENT (Both)

Goal

Use weaknesses to guide AI

Step 1 — Inject into Prompt

"User is weak in recursion and loop logic."

Step 2 — AI Behavior

- Gives more hints on weak areas
 - Focuses teaching
-

TASK 8.7 — DYNAMIC DIFFICULTY (Advanced)

Logic

If user improves:

- increase difficulty

If struggling:

- reduce difficulty
-

TASK 8.8 — TESTING (Both)

Test Cases

- Beginner user
 - Pro user
 - Weakness detection
 - Prompt variation
-

Expected

- ✓ AI adapts correctly
 - ✓ Weakness tracked
 - ✓ API works
-

TASK 8.9 — EDGE CASES

- New user (no weakness data)
 - Mixed skill levels
 - Incorrect AI detection
-

BLOCK 8 DONE CHECKLIST

- ✓ User level stored
 - ✓ Prompt changes dynamically
 - ✓ Weakness tracked
 - ✓ Weakness API working
 - ✓ AI personalization working
-



COMMON MISTAKES

- Static prompt → ❌
 - No weakness tracking → ❌
 - Overloading prompt → ❌
 - Ignoring user level → ❌
-



PART 10 — BLOCK 9: CACHING & RATE LIMITING



BLOCK 9: PERFORMANCE + PROTECTION LAYER

Goal

By the end of this block:

- Frequently used data is **cached (Redis)**
 - Expensive AI calls are **protected**
 - System prevents **abuse and overload**
 - APIs respond **fast even under load**
-



CORE IDEA

Without this:

- System becomes slow ❌
- AI costs/resources explode ❌
- Users can spam APIs ❌

With this:

- Fast responses ✓
- Stable system ✓
- Controlled usage ✓



TASK 9.1 — REDIS CACHE SETUP (Pavan)

Step 1 — Add Dependency

Add Redis starter dependency to services needing caching

Step 2 — Configure Redis

application.yml:

```
spring:  
  redis:  
    host: localhost  
    port: 6379
```

Step 3 — Enable Caching

Add in main class:

```
@EnableCaching
```



TASK 9.2 — CACHE STRATEGY DESIGN (Pavan)



What to Cache

1. Topics list
 2. Problems list
 3. Heatmap data
 4. User profile
-

What NOT to Cache

- AI responses (dynamic)
 - Authentication data
-

TASK 9.3 — IMPLEMENT CACHE (Pavan)

Step 1 — Add Annotation

```
@Cacheable(value = "topics")
public List getTopics() {
    return repository.findAll();
}
```

Step 2 — Cache Eviction

```
@CacheEvict(value = "topics", allEntries = true)
```

Why?

Keeps cache fresh

TASK 9.4 — REDIS KEY DESIGN (Kruthika)

Goal

Avoid conflicts

Pattern

topic:list
user:123:profile
user:123:heatmap

Important

Always prefix keys

TASK 9.5 — RATE LIMITING (Kruthika)

Goal

Prevent API abuse

Tool

Use Bucket4j

Rule

- AI APIs → 5 requests/min per user
 - Other APIs → 20 requests/min
-

Step 1 — Gateway Filter

Check:

- userId
 - request count
-

Step 2 — Reject if limit exceeded

Return:

HTTP 429



TASK 9.6 — IMPLEMENT FILTER

Inside Gateway:

```
if (requests > limit) {  
    return 429;  
}
```



TASK 9.7 — REDIS + RATE LIMIT (Advanced)

Store counters in Redis

Why:

- distributed system support
-



TASK 9.8 — PERFORMANCE OPTIMIZATION (Pavan)

Use:

- Async calls
 - Virtual Threads
 - Non-blocking APIs
-

Avoid:

- Blocking DB calls
 - Heavy loops
-



TASK 9.9 — TESTING (Both)

Test Cases

- Normal usage
 - High traffic
 - Rate limit exceed
 - Cache hit/miss
-

Expected

- ✓ Fast response
 - ✓ Rate limit working
 - ✓ No crashes
-



TASK 9.10 — MONITORING (Optional Advanced)

Track:

- API latency
 - Cache hits
 - Rate limit triggers
-



BLOCK 9 DONE CHECKLIST

- ✓ Redis working
- ✓ Cache implemented
- ✓ Cache eviction working

- ✓ Rate limiting working
 - ✓ System stable under load
-



COMMON MISTAKES

- Caching everything → ✗
 - No eviction → ✗
 - Global rate limit only → ✗
 - Not using Redis for scaling → ✗
-



PART 11 — BLOCK 10: PRODUCTION READY (DOCKER + FINAL SETUP)



BLOCK 10: PRODUCTION DEPLOYMENT



Goal

By the end of this block:

- All services run inside **Docker containers**
 - Entire system runs using **one command**
 - Backend is **environment-independent**
 - Swagger is unified and accessible via Gateway
-



FINAL ARCHITECTURE

Your system will run as:

- eureka-server
- api-gateway
- auth-service
- ai-service
- topic-service
- progress-service

- mysql
- redis
- ollama

👉 All inside Docker



TASK 10.1 — DOCKERFILE (Pavan)



Goal

Package each service into container

Step 1 — Create Dockerfile in each service

Dockerfile:

```
FROM openjdk:21-jdk-slim
```

```
WORKDIR /app
```

```
COPY target/*.jar app.jar
```

```
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Step 2 — Build JAR

Run:

```
mvn clean package
```

Step 3 — Build Docker Image

```
docker build -t auth-service .
```

Repeat for all services



TASK 10.2 — DOCKER COMPOSE (Kruthika)

Goal

Run entire system together

Step 1 — Create docker-compose.yml

Define services:

- mysql
 - redis
 - ollama
 - eureka-server
 - api-gateway
 - auth-service
 - ai-service
 - topic-service
 - progress-service
-

Step 2 — Example Structure

```
mysql:
image: mysql:latest
environment:
MYSQL_ROOT_PASSWORD: root
```

```
redis:
image: redis:latest
```

```
ollama:
image: ollama/ollama
```

Step 3 — Add App Services

Each service:

- build: .
 - depends_on:
 - ports
-

Step 4 — Run

docker-compose up

Result

✓ Entire backend starts

TASK 10.3 — ENV CONFIG (Both)

Goal

Remove hardcoding

Step 1 — Use Environment Variables

DB_URL
DB_USER
DB_PASSWORD

Step 2 — Inject in application.yml

TASK 10.4 — SERVICE COMMUNICATION (Pavan)

Important

Inside Docker:

- Use service names (not localhost)

Example:

jdbc:mysql://mysql:3306/petallogic

TASK 10.5 — HEALTH CHECKS (Kruthika)

Add endpoints

/actuator/health

Why?

Monitor services

TASK 10.6 — SWAGGER UNIFICATION (Both)

Goal

Single API UI

Step 1 — Enable Swagger in all services

Step 2 — Gateway aggregates

Access

<http://localhost:8080/swagger-ui>

Result

✓ All APIs visible in one place



TASK 10.7 — LOGGING (Pavan)

Add logging

```
log.info("User login success")
```

Use log levels

- INFO
 - ERROR
 - DEBUG
-



TASK 10.8 — FINAL TESTING (Both)

Test everything:

- Auth
 - AI
 - Topics
 - Analytics
 - Rate limit
 - Cache
-

Also test:

- Container restart
 - DB restart
 - Failure recovery
-



TASK 10.9 — PRODUCTION HARDENING

Add:

- HTTPS (later)
 - Secrets management
 - Backup strategy
-



TASK 10.10 — FINAL CHECKLIST

- ✓ All services containerized
 - ✓ Docker compose working
 - ✓ Env variables used
 - ✓ Gateway routing works
 - ✓ Swagger unified
 - ✓ System stable
-



COMMON MISTAKES

- Using localhost inside Docker → ✗
 - Hardcoding credentials → ✗
 - Not testing restart → ✗
 - Ignoring logs → ✗
-



FINAL RESULT

If you followed everything:

You now have:

- ✓ Microservices architecture
 - ✓ Secure backend (JWT + Argon2)
 - ✓ AI-powered engine
 - ✓ Personalized learning system
 - ✓ Analytics + gamification
 - ✓ High performance (Redis + caching)
 - ✓ Production-ready deployment
-



WHAT YOU BUILT

This is not just a backend.

This is:

- 👉 AI Learning Platform
 - 👉 Scalable Microservices System
 - 👉 Production-grade Startup Backend
-



FINAL STEP

Now:

- 👉 Start frontend (React)
 - 👉 Connect APIs
 - 👉 Deploy to cloud
-